P9 PRESENTER CUE CARDS · ฉบับภาพสไลด์จริง

สคริปต์พูดนำเสนอ P9 พร้อมรูปภาพสไลด์จริง (19 หน้า)

ออกแบบให้เป็น Presenter Cue Cards หน้าต่อหน้า (1 Slide ต่อ 1 แผ่น A4)
เห็นภาพสไลด์ชัดเจน จุดที่ต้องชี้บนจอ และคำพูดสั้นกระชับเข้าใจง่าย ไม่เยิ่นเย้อ

- 🖼️ มีภาพสไลด์จริงติดครบทุกหน้า
- ⚡ สคริปต์สั้นกระชับ 2-4 บรรทัดจบ
- 👉 จุดชี้นิ้วบนจอชัดเจน (Visual Cues)
- 💎 พิสูจน์ Composition 5 คู่แท้

วิชา: Object-Oriented Analysis and Design (OOAD)
โปรเจกต์: ระบบร้านข้าวต้มอารมณ์ดี (Main + 16 Model Classes)

ฉบับนำเสนอ: พฤศจิกายน 2026
เวลาพูดรวม: ~5-7 นาที (กระชับ เข้าประเด็น)

หน้า 1: หน้าปก P9 (Java OOP)

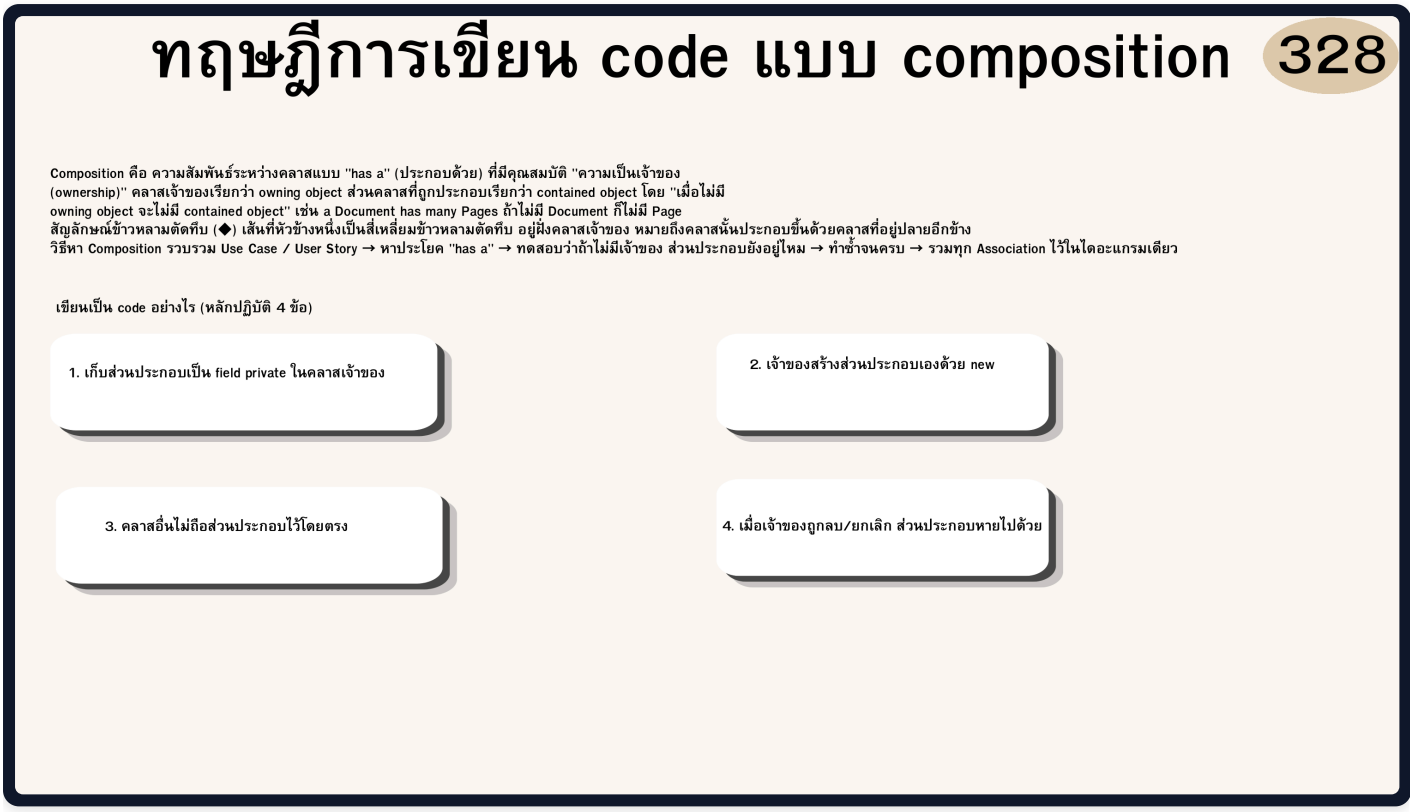


👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
สบตาอาจารย์ ผายมือไปที่หน้าจอ พูดด้วยน้ำเสียงมั่นใจ สบายๆ

💡 **ทริคการพูด:**
พูดสั้นๆ 15 วินาที เพื่อเปิดประเด็น ไม่ต้องอ่านรายละเอียดยาว

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

สวัสดีครับอาจารย์และเพื่อนๆ ทุกคน วันนี้กลุ่มเราจะมาพรีเซนต์ **Phase 9: Java OOP Implementation** ของระบบร้านข้าวต้มอารมณ์ดีครับ โดยเรานำการออกแบบจาก P1 ถึง P8 มาเขียนเป็นโปรแกรมจริงด้วย **Java OOP พื้นฐาน 16 คลาสโมเดล** โดยไม่พึ่งพา Framework ภายนอกเลยครับ



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้ที่เครื่องหมายข้าวหลามตัดทึบ (◆) และชี้ที่กฎ 4 ข้อ

💡 **ทริคการพูด:**
ย้ำคำว่า 'ความเป็นเจ้าของร่วมตาย' อาจารย์จะเข้าใจทันที

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

หน้านี้คือทฤษฎี Composition ครับ สรุปให้เข้าใจง่ายที่สุดคือ **'ความเป็นเจ้าของร่วมตาย'** หมายถึง *ถ้าไม่มีตัวแม่ ตัวลูกที่เป็นส่วนประกอบย่อยก็อยู่ไม่ได้ ต้องหายไปด้วยกันทันที* สัญลักษณ์คือข้าวหลามตัดทึบ (◆) อยู่ฝั่งตัวแม่เสมอ เช่น Order กับ OrderItem และมีกฎโค้ด 4 ข้อคือ: แม่เก็บลูกเป็น private, แม่ new ลูกเอง, คนอื่นห้ามแตะลูกตรงๆ, และแม่ตายลูกต้องตายด้วยครับ

329

ตัวอย่างของระบบเรา (src/model/Order.java)

Order ◆ OrderItem (ข้าวต้มอารมณ์ดี) 📋 ก๊อปโค้ด

```
public class Order {
    private ArrayList<OrderItem> items; // Composition (ลูกตายตามแม่)
    private Table table;                // Association (โต๊ะยังคงอยู่)

    public OrderItem addItem(Menu menu, int qty, String spice, String note) {
        OrderItem item = new OrderItem(orderId + "-ITM-" + itemNum, menu, qty,
spice, note);
        items.add(item); // ส่วนประกอบเกิดในเจ้าของ
        return item;
    }

    public void destroy() {
        for (int i = 0; i < items.size(); i++)
            items.get(i).destroy(); // สั่งทำลาย OrderItem ทั้งหมด
        items.clear();
        table = null;               // โต๊ะไม่ถูกทำลาย (ยังคงอยู่รับลูกค้าใหม่)
        destroyed = true;
    }
}
```

Order ◆ OrderItem : ยกเลิก/ลบออเดอร์แล้ว OrderItem หายไปพร้อมกัน แต่ Table ยังคงอยู่

👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้เปรียบเทียบ: items (Composition) vs table (Association)

💡 **ทริคการพูด:**
ชี้จุดต่างชัดเจน: items หายไป vs table ยังอยู่

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

เทียบกับโค้ดจริงใน `Order.java` ครับ บรรทัดบนสุด `items` คือ Composition สังเกตว่าในเมธอด `destroy()` พอสั่งยกเลิกออเดอร์ รายการอาหารจะถูกล้างทิ้งทันที (ลูกตายตามแม่) แต่ในขณะเดียวกัน `table` เป็นแค่ Association พอยกเลิกออเดอร์ เราแค่ปลดการเชื่อมโยง แต่โต๊ะอาหารจริงยังคงอยู่ในระบบ ไม่พังตามออเดอร์ครับ

Text Mode Simulator
ร้านข้าวต้มอารมณ์ดี (FR1–FR8)

👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
กวาดสายตามองผู้ฟัง แล้วเกริ่นเข้าสู่การสาธิตหน้าจอ

💡 **ทริคการพูด:**
พูดประโยคเดียวสั้นๆ 5 วินาที แล้วกดเปลี่ยนสไลด์ถัดไป

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**
ต่อไปเป็นส่วนสาธิตการทำงานจริงผ่าน **Text Mode Simulator** ซึ่งรองรับการทำงานครบทั้ง 8 Use Cases (FR1 ถึง FR8) ครับ

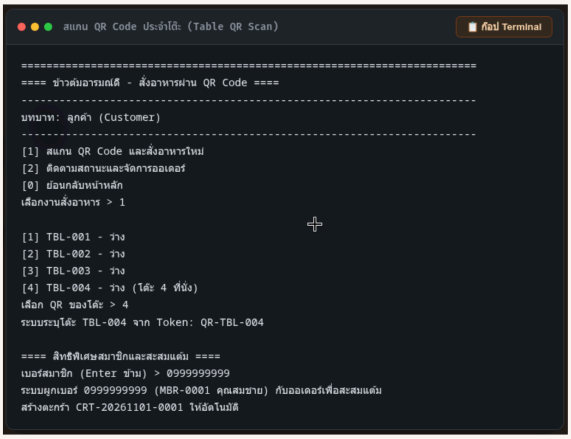
หน้า 5 (331): FR1 (1.1 สแกน QR โต๊ะ & 1.2 เลือกเมนู ปรับระดับเผ็ด)

⏱️ ~20-30 วินาที

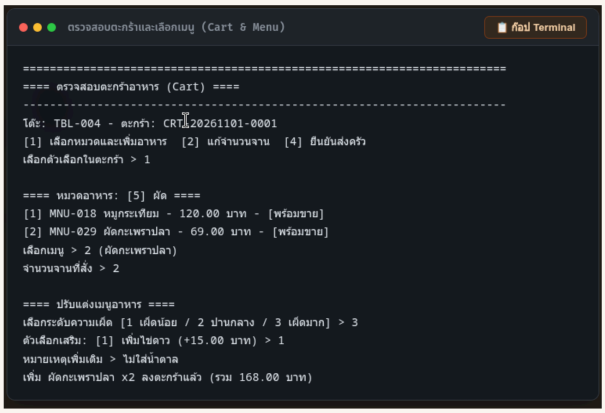
FR1

331

1.1 สแกน QR ประจำโต๊ะ และเชื่อมโยงสมาชิกสะสมแต้ม



1.2 เลือกเมนู ปรับแต่งระดับเผ็ด และเพิ่มตัวเลือกเสริม



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้กล่องซ้ายตรงการผูกเบอร์สมาชิก -> ชี้กล่องขวาตรงยอด 168 บาท

💡 **ทริคการพูด:**
พูดสั้น ชี้ตาม 2 สเต็ป: ผูกสมาชิก -> สั่งอาหารพร้อมท็อปปิ้ง

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

เริ่มที่ **FR1** ครับ ด้านซ้าย (1.1) ลูกค้าสแกน QR โต๊ะ 4 กรอกเบอร์โทร ระบบจะค้นหาและผูกกับสมาชิกคุณสมชาย และสร้างตะกร้าให้อัตโนมัติ ส่วนด้านขวา (1.2) ลูกค้าเลือกเมนูกะเพราปลา 2 จาน เลือกระดับความเผ็ด และเพิ่มท็อปปิ้งไข่ดาว ระบบคำนวณยอดรวมได้ 168 บาทครับ

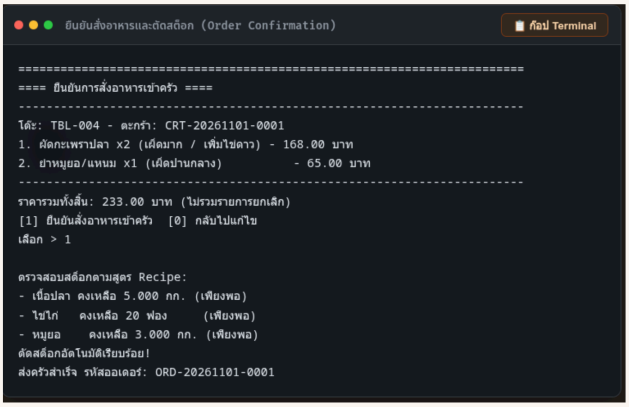
หน้า 6 (332): FR1 (1.3 ตรวจสต็อกตัดของ & 1.4 ติดตามสถานะออเดอร์)

⏱️ ~20-30 วินาที

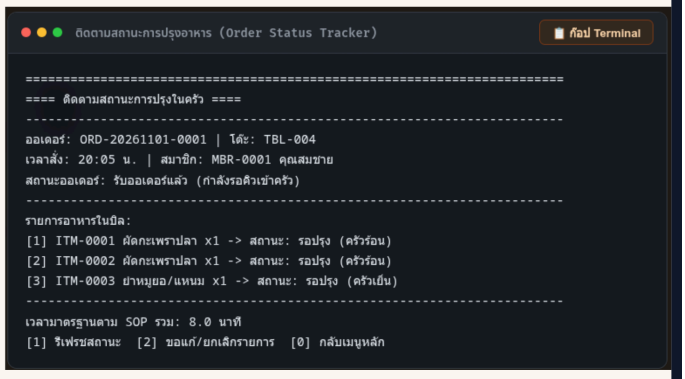
FR 1

332

1.3 ตรวจสต็อกตาม Recipe ตัดสต็อก และยืนยันส่งครัว



1.4 เลือกเมนู ปรับแต่งระดับเผ็ด และเพิ่มตัวเลือกเสริม



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้กล่องซ้ายตรงตรวจสต็อก Recipe -> ชี้กล่องขวาตรงสถานะครัวร้อน/ครัวเย็น

💡 **ทริคการพูด:**
เน้นคำว่า Two-Pass Check (ตรวจให้ครบก่อน แล้วค่อยตัดจริง)

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

ด้านซ้าย (1.3) เมื่อกดยืนยันออเดอร์ยอด 233 บาท ระบบจะใช้กลไก **Two-Pass Stock Check** คือตรวจสต็อกวัตถุดิบทุกอย่างให้ครบก่อน เมื่อผ่านครบจึงตัดสต็อกจริงและส่งเข้าครัว ส่วนด้านขวา (1.4) คือหน้าจอติดตามสถานะ โดยระบบจะแยกรายการอาหารส่งไปยังครัวร้อนและครัวเย็นตามเวลา SOP 8 นาทีครับ

หน้า 7 (333): FR2 (2.1 รายการคู่มือ SOP & 2.2 กำหนดขั้นตอนย่อย)

⏱️ ~20-30 วินาที



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้กล่องซ้ายตรงรายการคู่มือ -> ชี้กล่องขวาตรงเวลารวม 2.0 นาที

💡 **ทริคการพูด:**
ชี้ว่าคู่มือมีขั้นตอนย่อยรวมเวลาได้อัตโนมัติ

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

FR2 การจัดการคู่มือ SOP: ด้านซ้าย (2.1) ผู้จัดการสามารถดูคู่มือการทำอาหารทั้งหมดในร้าน ส่วนด้านขวา (2.2) สามารถกำหนดขั้นตอนย่อย `SOPStep` ได้ เช่น เมนูชาใหม่ มี 2 ขั้นตอน รวม 2 นาที เมื่อขั้นตอนพร้อมแล้วก็กดเผยแพร่ เพื่อส่งต่อให้ครัวนำไปเป็นมาตรฐานการทำอาหารครับ

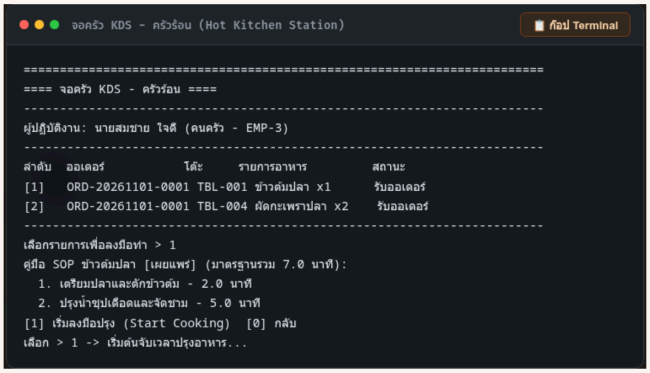
หน้า 8 (334): FR3 (3.1 ครัว KDS รับคิว & 3.2 บันทึกเวลาเทียบ SOP)

⏱️ ~20-30 วินาที

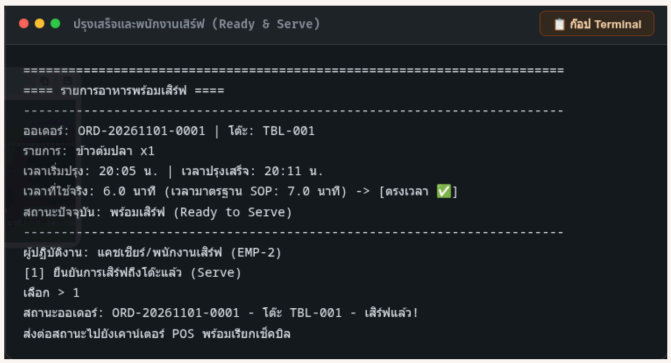
FR3

334

3.1 คนครัวรับคิว เปิดดู SOP และเริ่มจับเวลาปรุง



3.2 บันทึกเวลาปรุงจริงเทียบ SOP และยืนยันการเสิร์ฟ



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้กล่องซ้ายตรงเริ่มปรุง -> ชี้กล่องขวาตรงเวลาจริง 6 นาที vs เป้าหมาย 7 นาที

💡 **ทริคการพูด:**
ชี้จุดเปรียบเทียบเวลา: ใช้จริง 6 นาที เร็วกว่ามาตรฐาน 7 นาที

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

FR3 หน้าจอครัว KDS: ด้านซ้าย (3.1) คนครัวเปิดดูคิวอาหาร ดูขั้นตอนและเวลามาตรฐาน 7 นาที แล้วกดเริ่มปรุง ส่วนด้านขวา (3.2) เมื่อทำเสร็จ ระบบจับเวลาจริงได้ 6 นาที ซึ่งเร็วกว่ามาตรฐาน SOP แล้วพนักงานก็กดยืนยันการเสิร์ฟอาหารไปที่โต๊ะครับ

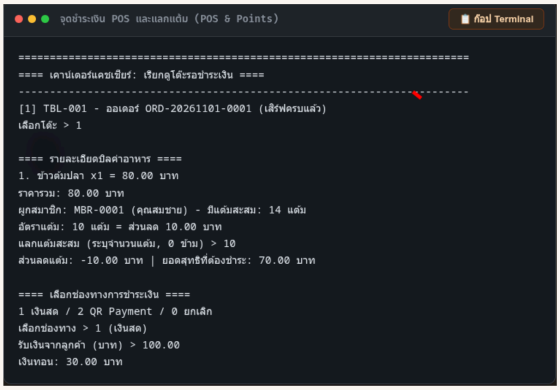
หน้า 9 (335): FR4 (4.1 ตรวจบิล แลกแต้ม & 4.2 ออกใบเสร็จ สะสมแต้ม)

⏱️ ~20-30 วินาที

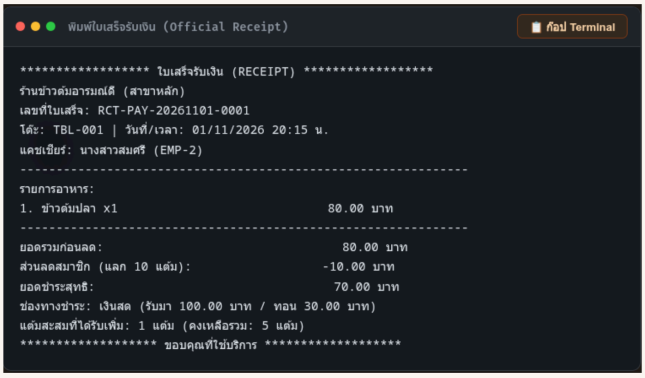
FR4

335

4.1 ตรวจสอบบิลโต๊ะอาหาร แลกแต้มส่วนลด และรับชำระเงิน



4.2 พิมพ์ใบเสร็จ คำนวณเงินทอน และสะสมแต้มสมาชิกใหม่



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้กล่องซ้ายตรงแลก 10 แต้ม -> ชี้กล่องขวาตรงใบเสร็จและเงินทอน 30 บาท

💡 **ทริคการพูด:**
ชี้ตัวเลขชัดๆ: ยอด 80 - ส่วนลด 10 = จ่าย 70 ทอน 30 ได้แต้มเพิ่ม 1

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

FR4 การคิดเงิน: ด้านซ้าย (4.1) แคชเชียร์เรียกบิลโต๊ะ 1 ยอด 80 บาท ลูกค้าขอแลก 10 แต้มเป็นส่วนลด 10 บาท เหลือยอดชำระ 70 บาท จ่ายเงินสด 100 บาท ทอน 30 บาท ด้านขวา (4.2) ระบบพิมพ์ใบเสร็จและคำนวณแต้มสะสมเพิ่มให้อีก 1 แต้มทันทีครับ

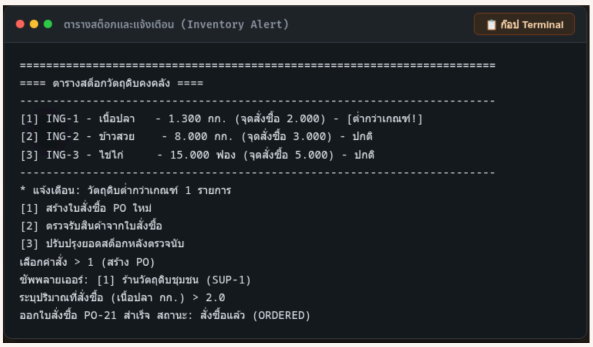
หน้า 10 (336): FR5 (5.1 ตรวจสต็อก ออก PO & 5.2 ตรวจรับเข้าคลัง Audit Log)

⏱️ ~20-30 วินาที

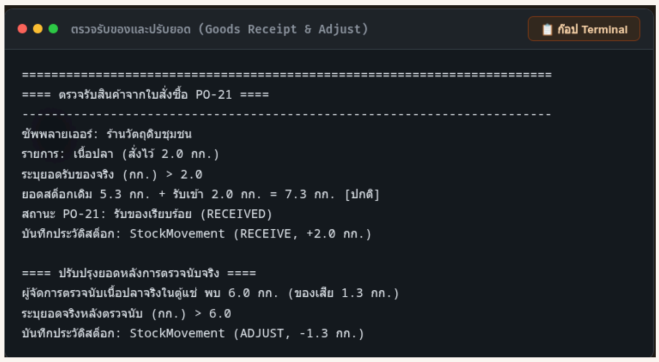
FR5

336

5.1 ตรวจสอบสต็อก แจ้งเตือน และออกใบสั่งซื้อ PO



5.2 ตรวจรับสินค้าเข้าคลัง และบันทึก Audit Log การปรับยอด



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้กล่องซ้ายตรงแจ้งเตือนสต็อกต่ำ -> ชี้กล่องขวาตรง StockMovement RECEIVE & ADJUST

💡 **ทริคการพูด:**
ชี้ที่คำว่า Audit Log ในกล่องขวา

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

FR5 สต็อกและใบสั่งซื้อ: ด้านซ้าย (5.1) ระบบแจ้งเตือนเนื้อปลาเหลือ 1.3 กก. ต่ำกว่าเกณฑ์ ผู้จัดการจึงออกใบ PO-21 สั่ง 2 กก. ด้านขวา (5.2) ตรวจรับของเข้าคลัง สต็อกเพิ่มเป็น 7.3 กก. และบันทึกประวัติ และเมื่อตรวจนับจริงพบของเสีย จึงปรับยอดเหลือ 6 กก. โดยมี Audit Log บันทึกประวัติย้อนหลังครบถ้วนครับ

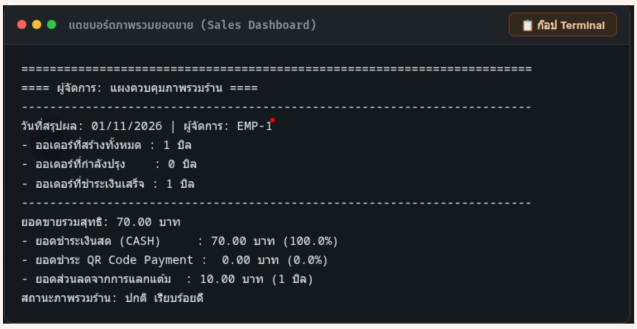
หน้า 11 (337): FR6 (6.1 สรุปยอดขาย & 6.2 อันดับขายดีและสถิติ SOP)

⏱️ ~20-30 วินาที

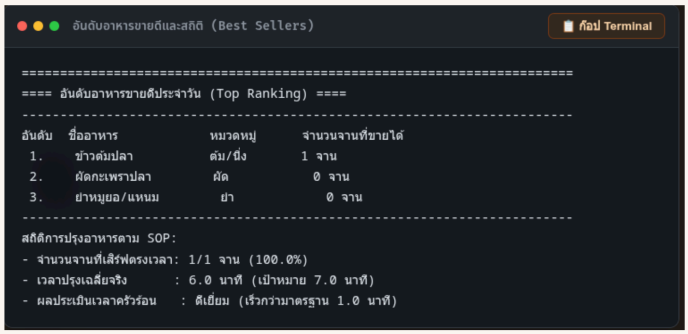
FR6

337

6.1 สรุปยอดขายรวม จำแนกช่องทางเงินสด/QR และการใช้แต้ม



6.2รายงานอันดับอาหารขายดี และสถิติความเร็วปรุงตาม SOP



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้กล่องซ้ายตรงแยกเงินสด/QR -> ชี้กล่องขวาตรงเสิร์ฟตรงเวลา 100%

💡 **ทริคการพูด:**
เน้นสถิติ: ขายดีอันดับ 1 + เสิร์ฟตรงเวลา 100%

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

FR6 รายงานภาพรวม: ด้านซ้าย (6.1) สรุปยอดขายประจำวันแยกช่องทางเงินสด 70 บาท และ QR 0 บาท ส่วนด้านขวา (6.2) แสดงเมนูขายดีอันดับ 1 คือข้าวต้มปลา และสถิติเสิร์ฟตรงเวลา 100% ประเมินผลระดับดีเยี่ยมครับ

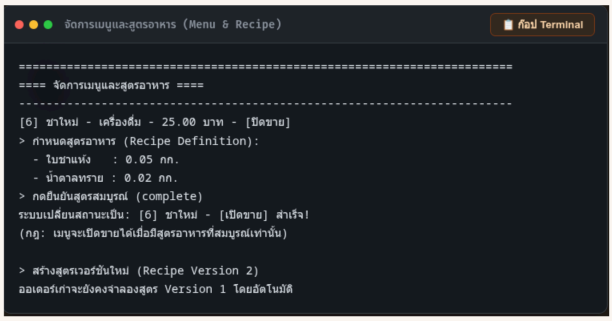
หน้า 12 (338): FR7 (จัดการสูตร) & FR8 (ระบบสมาชิกและกฎแต้ม)

⏱️ ~20-30 วินาที

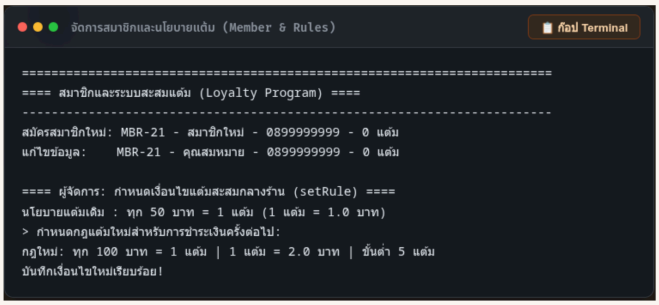
FR7+FR8

338

7.1 จัดการเปิด/ปิดการขายเมนูอาหารและสูตรอาหาร



8.1ระบบสมาชิกและการสะสมแต้มสมาชิก



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้กล่องซ้ายตรง Recipe Version 2 -> ชี้กล่องขวาตรงเปลี่ยนกฎแต้ม setRule()

💡 **ทริคการพูด:**
ชี้ว่าระบบรองรับสูตรอาหารหลายเวอร์ชันและเปลี่ยนกฎแต้มได้

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

FR7 และ FR8: ด้านซ้าย (7.1) เปิดขายเมนูชาใหม่ และรองรับสูตรหลายเวอร์ชัน โดยออเดอร์เก่ายังคงยึดสูตร Version 1 เดิม ไม่เพี้ยน ส่วนด้านขวา (8.1) ระบบสมาชิก ผู้จัดการสามารถเปลี่ยนกฎแต้มกลางร้านได้ เช่น ทุก 100 บาทได้ 1 แต้ม มีผลทันทีในการจ่ายเงินครั้งต่อไปครับ

หน้า 13 (339): FR6 (แผงควบคุมสถิติร้านค้าสำหรับผู้บริหาร)

⏱️ ~20-30 วินาที



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้ภาพรวมแดชบอร์ดสรุปความเรียบร้อยของร้าน

💡 **ทริคการพูด:**
พูดสั้นๆ 10 วินาที เพื่อเชื่อมเข้าสู่หมวดถัดไป

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

หน้านี้คือแผงควบคุมรวมของ FR6 อีกมุมมองหนึ่ง ซึ่งช่วยให้ผู้จัดการเห็นภาพรวมทั้งยอดขาย สต็อก และประสิทธิภาพของพนักงานในหน้าจอเดียวครับ

ตัวอย่าง Composition 5 คู่แท้

👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ปรับน้ำเสียงให้หนักแน่น มั่นใจ เพราะเป็นหัวใจของ OOAD

💡 **ทริคการพูด:**
ย้ำว่าเรามี Composition ถึง 5 คู่แท้

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**
ในส่วนสุดท้ายนี้ เราจะมา **พิสูจน์ Composition ครบทั้ง 5 คู่แท้** ที่เกิดขึ้นจริงในระบบเพื่อยืนยันความถูกต้องของ Object Design ครับ

Composition ① Order ◆ OrderItem ยกเลิกออเดอร์แล้วรายการหายด้วย

340

ค้นหาออเดอร์ + ยกเลิก (Composition) — ก๊อป Terminal

```
กรอกเลขออเดอร์ เช่น ORD-001 : ORD-001
ประเภท : [DINE-IN] รับประทานที่ร้าน | โต๊ะ : TBL-004
ชื่อลูกค้า : คุณสมชาย (MBR-0001) | เวลาสั่ง : 20:05 น.
รายการอาหาร :
1. ข้าวต้มปลา x1   80.00 บาท [รอปรุง]
2. ผัดกะเพราปลา x1 69.00 บาท [รอปรุง]
ยอดรวม : 149.00 บาท
------------------------------------------------------------
[1] ยกเลิก/ทำลายออเดอร์นี้ (destroy)  [0] กลับ
เลือก : 1
ยกเลิกออเดอร์ ORD-001 สำเร็จ
จำนวนรายการอาหารในออเดอร์: 2 -> 0 รายการ (OrderItem ถูกลบตามหลัก Composition)
```

โค้ดจริง · src/model/Order.java — ก๊อปโค้ด

```
public void destroy() {
    // 1. สั่งทำลายส่วนประกอบลูก (OrderItem) ทั้งหมดตามหลัก Composition
    for (int i = 0; i < items.size(); i++)
        items.get(i).destroy();
    items.clear();

    // 2. เคลียร์ความสัมพันธ์แบบ Association (โต๊ะและลูกค้ายังคงอยู่ในระบบ)
    table = null;
    customer = null;
    payment = null;
    destroyed = true;
}
```

คู่ที่พิสูจน์
Order (เจ้าของ) ประกอบด้วย **OrderItem** (ส่วนย่อย)

ก่อนยกเลิก
ออเดอร์ ORD-001 ถือ OrderItem จำนวน 2 รายการ

โค้ดที่ทำงาน
`order.destroy()` → `item.destroy()` และ `items.clear()`

หลังยกเลิก
เหลือ **0 รายการ** = ส่วนประกอบย่อยหายไปพร้อมเจ้าของทันที

ส่วน TABLE และ CUSTOMER
ยังอยู่ครบในระบบ เพราะความสัมพันธ์เป็นแค่ Association "อ้างถึง"

👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้ที่จำนวน 2 -> 0 รายการ และชี้โค้ด items.clear()

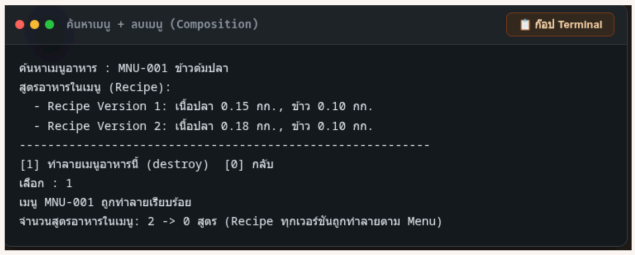
💡 **ทริคการพูด:**
เน้นตัวเลข: จาก 2 รายการ เหลือ 0 รายการทันที

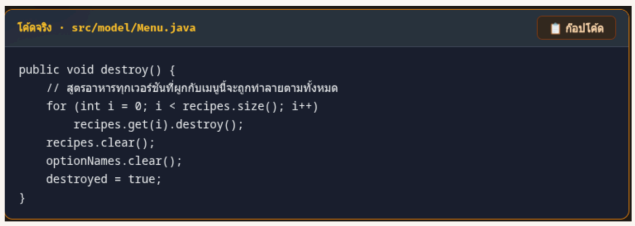
💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

คู่ที่ 1: Order เป็นเจ้าของ OrderItem
ออเดอร์มีรายการอาหารอยู่ 2 รายการ พอกดยกเลิกออเดอร์ `order.destroy()` รายการอาหารทั้งหมดถูกลบหายตามไปทันที เหลือ 0 รายการ พิสูจน์ว่าลูกตายตามแม่ ส่วนโต๊ะและลูกค้ายังคงอยู่ในระบบตามเดิมครับ

Composition ② Menu ◆ Recipe เมนูเป็นเจ้าของสูตรอาหาร

341







👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้สูตร 2 เวอร์ชัน ลดเหลือ 0 สูตร และเน้นว่าวัตถุดิบยังอยู่

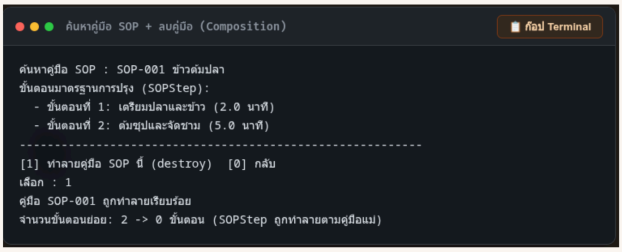
💡 **ทริคการพูด:**
เน้น: สูตรตายตามเมนู แต่วัตถุดิบยังอยู่

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

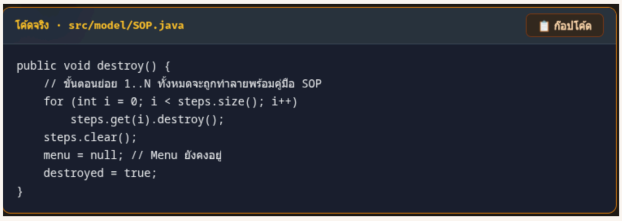
คู่ที่ 2: Menu เป็นเจ้าของ Recipe (สูตรอาหาร)
เมนูข้าวต้มปลามีสูตรอาหารอยู่ 2 เวอร์ชัน พอลบเมนู `menu.destroy()` สูตรอาหารทั้ง 2 เวอร์ชันถูกลบหายตามเมนูไปทันที เหลือ 0 สูตร แต่วัตถุดิบในคลังยังคงอยู่ เพราะเป็นแค่ Association ครับ

Composition ③ SOP ◆ SOPStep คู่มือเป็นเจ้าของขั้นตอนมาตรฐาน

342







👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้ขั้นตอนย่อย 2 ขั้นตอน ลดเหลือ 0 ขั้นตอน

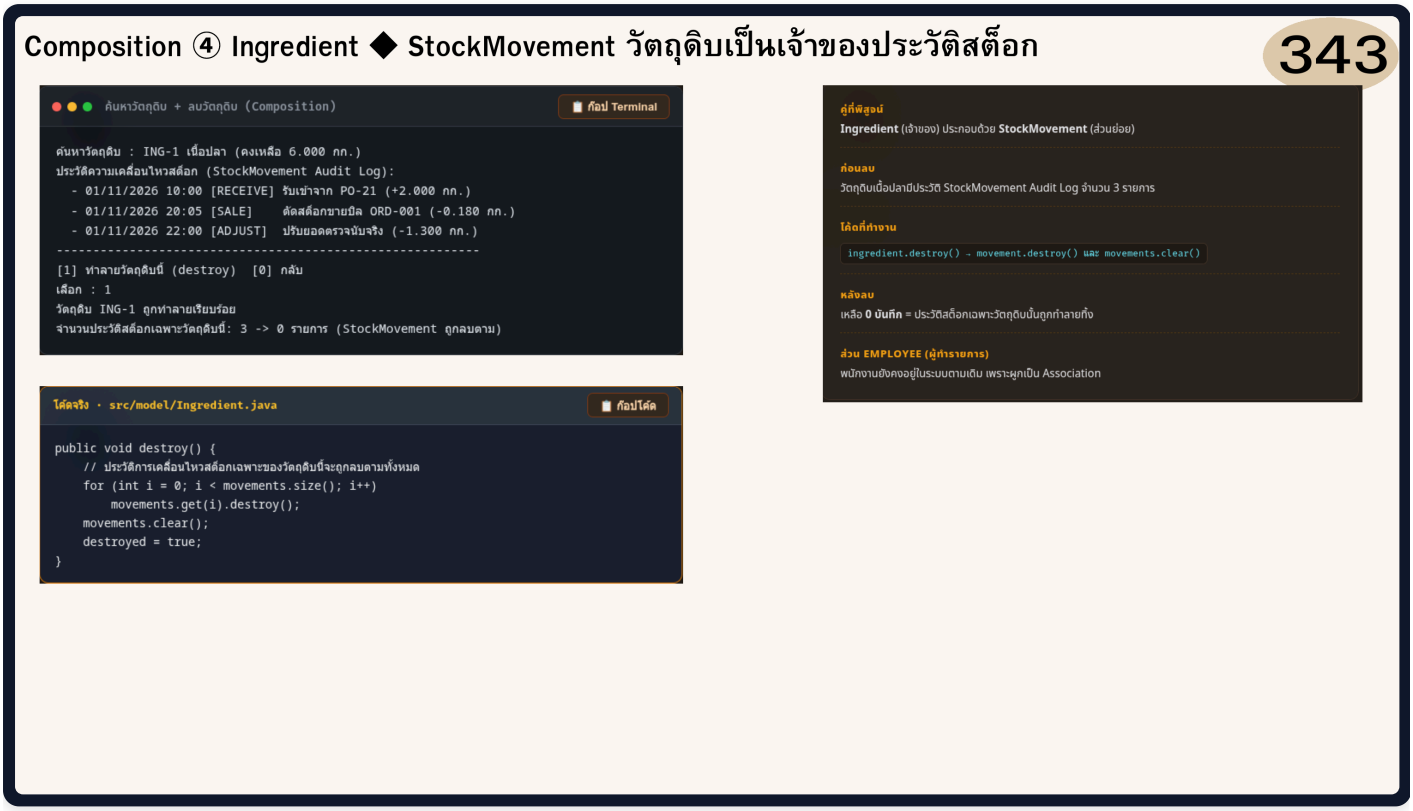
💡 **ทริคการพูด:**
เน้น: ขั้นตอนย่อยหายไปพร้อมคู่มือ

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

คู่ที่ 3: SOP เป็นเจ้าของ SOPStep (ขั้นตอนย่อย)
คู่มือ SOP ข้าวต้มปลามีขั้นตอนย่อย 2 ขั้นตอน พอลบคู่มือ `sop.destroy()` ขั้นตอนย่อยทั้งหมดหายไปพร้อมคู่มือทันที เหลือ 0 ขั้นตอนครับ

หน้า 18 (343): Composition ④ Ingredient ◆ StockMovement

⏱️ ~20-30 วินาที



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้ประวัติสต็อก 3 รายการ ลดเหลือ 0 บันทึก

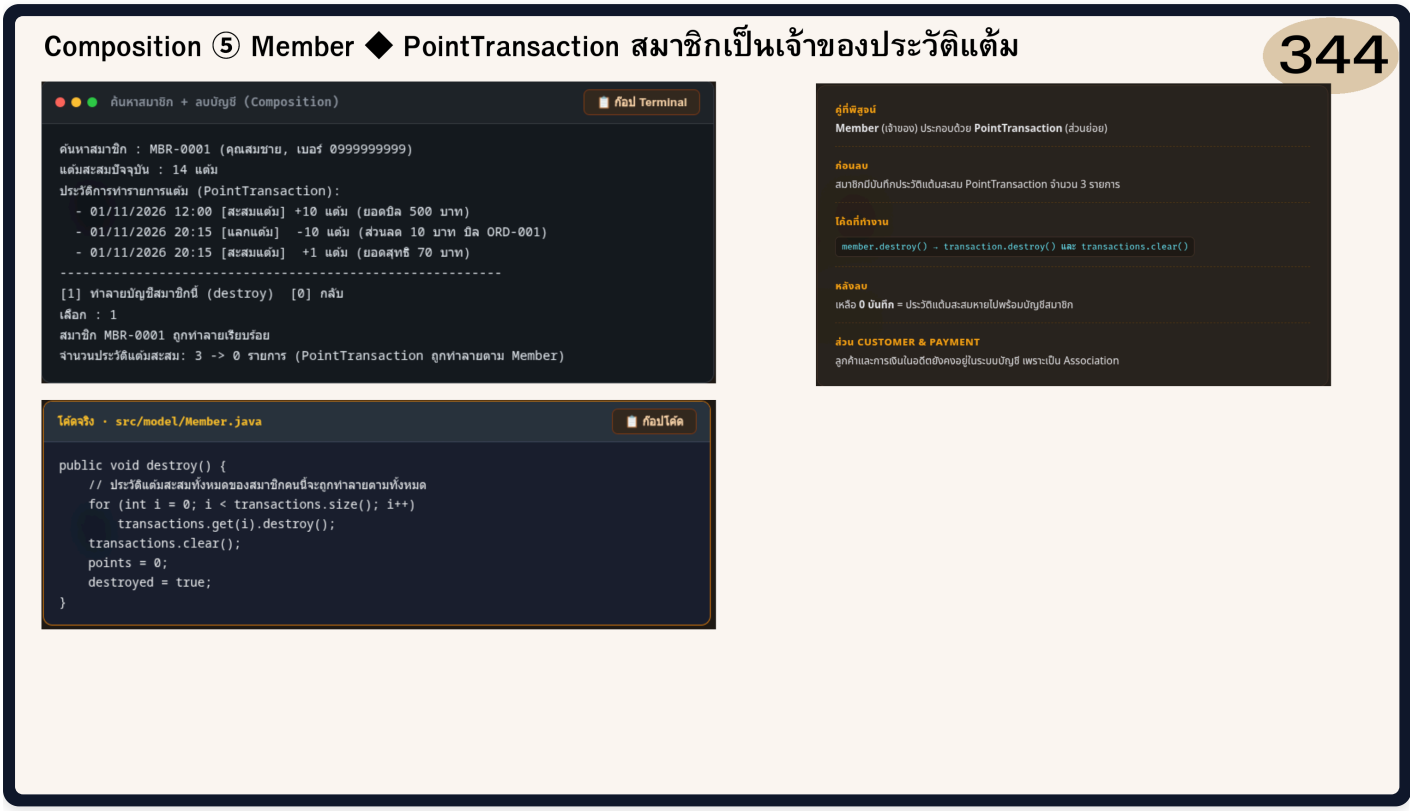
💡 **ทริคการพูด:**
เน้น: ประวัติสต็อกผูกติดกับตัววัตถุดิบ

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

คู่ที่ 4: Ingredient เป็นเจ้าของ StockMovement (ประวัติสต็อก)
วัตถุดิบเนื้อปลามีประวัติสต็อกอยู่ 3 รายการ พอลบวัตถุดิบ `ingredient.destroy()` ประวัติสต็อกเฉพาะของชิ้นนั้นถูกลบหายตามไปด้วยทันที เหลือ 0 รายการครับ

หน้า 19 (344): Composition ⑤ Member ◆ PointTransaction

⏱️ ~20-30 วินาที



👉 **จุดที่ต้องชี้บนจอ (Visual Cue):**
ชี้ประวัติแต้ม 3 รายการ เหลือ 0 บันทึก แล้วสบตาอาจารย์เพื่อขอบคุณ

💡 **ทริคการพูด:**
จบอย่างมั่นใจ ยิ้มและขอบคุณผู้ฟัง

💬 **คำพูดนำเสนอ (สั้น กระชับ พูดตามนี้ได้เลย):**

คู่ที่ 5: Member เป็นเจ้าของ PointTransaction (ประวัติแต้ม)
สมาชิกมีประวัติแต้ม 3 รายการ พอลบบัญชี `member.destroy()` ประวัติแต้มสะสมทั้งหมดหายไปพร้อมบัญชีทันทีครับ

ทั้ง 5 คู่นี้พิสูจน์ให้เห็นว่าระบบของเราปฏิบัติตามหลัก Clean OOP และ OOAD อย่างถูกต้องสมบูรณ์แบบครับ กลุ่มเราขอจบการนำเสนอเพียงเท่านี้ ขอบคุณครับ

🛡️ คลังคำถามปราบเซียนสำหรับเตรียมตอบอาจารย์ (Q&A Defense)

❓ Q1: ใน Java ไม่มีคำสั่ง delete เหมือนภาษา C++ แล้วเราทำ Composition อย่างไร?

แนวทางการตอบ: "ใน Java หน่วยความจำถูกดูแลโดย Garbage Collector ครับ เราจึงทำ Composition ผ่าน Life-cycle Method คือ `destroy()` โดยเมื่อ Object ตัวแม่ถูกทำลาย ตัวแม่จะเป็นคนสั่ง `clear()` และตัดการอ้างอิง Object ลูกทั้งหมด ทำให้ลูกกลายเป็นขยะและถูก Garbage Collector กวาดทิ้งไปพร้อมกันครับ"

❓ Q2: Two-Pass Stock Check ใน FR1 คืออะไร ทำไมต้องทำสองรอบ?

แนวทางการตอบ: "คือการแยกตรวจกับตัดสต็อกออกเป็น 2 รอบครับ รอบแรกวนตรวจวัตถุดิบทุกอย่างให้มั่นใจว่าพอครบ 100% ก่อน เมื่อผ่านครบแล้วรอบสองจึงค่อยตัดสต็อกจริง ป้องกันปัญหาหักชิ้นแรกไปแล้วของชิ้นถัดไปหมดครับ"

❓ Q3: ทำไม OrderItem ต้องเก็บราคาและสูตรของตัวเอง ทั้งที่มี Menu อยู่แล้ว?

แนวทางการตอบ: "เพื่อป้องกัน Historical Data Drift ครับ ถ้าร้านปรับราคาเมนูในวันพรุ่งนี้ บิลย้อนหลังที่ลูกค้าเคยสั่งไปเมื่อวานจะต้องไม่เปลี่ยนราคาตาม จึงต้อง Snapshot ค่าเก็บไว้ใน OrderItem ครับ"

❓ Q4: ทำไมความสัมพันธ์ของ Order กับ Table ถึงเป็นแค่ Association ไม่ใช่ Composition?

แนวทางการตอบ: "เพราะเรื่อง Lifecycle ครับ โต๊ะอาหารมีอยู่จริงในร้านมาก่อนออเดอร์ และเมื่อออเดอร์ถูกยกเลิกหรือจ่ายเงินเสร็จ โต๊ะก็ยังคงอยู่ที่ร้านรับลูกค้ารายใหม่ได้ ไม่ได้พังหรือหายไปตามออเดอร์ครับ"

❓ Q5: ทำไมโปรเจกต์นี้ถึงไม่มีการใช้ Framework เช่น Spring Boot?

แนวทางการตอบ: "เพราะวิชา OOAD ต้องการพิสูจน์ความเข้าใจในกลไกพื้นฐานของ OOP ครับ การใช้ Core Java ทำให้เห็นการทำงานของ Constructor, Encapsulation, Reference และ Garbage Collection ครบถ้วนทุกบรรทัดอย่างโปร่งใสครับ"